

Изискване към задачите е постигане поне на слаба гаранция при възникване на изключение. Приемете, че входните данни са коректни.

Задача 1. Run-length encoding (RLE) представлява кодиране на стрингове, при което последователности от еднакви символи се заменят с двойка от символа и броя повторения. Например следният стринг:

AAAABBBBCCCA

може да се представи с RLE по следния начин:

(4,A) (4,B) (3,C) (1,A)

Забележете, че в горното представяне числата са строго по-големи от нула и не може да има две съседни двойки съдържащи един и същ символ. Например това по-долу НЕ Е валидно представяне на горния низ:

(2,A) (2,A) (4,B) (0,B) (0,B) (0,B) (6,C) (-3,C) (1,A) (0,Z)

Да се напише клас RLEString, който представя низове чрез RLE по горе-описания начин. Класът трябва да може да получава C-style символен низ (const char*) и вътрешно да го съхранява като едносвързан списък от двойки от вида (число, символ).

За класа е достатъчно да се реализират следните операции:

- Конструктор с аргумент char const*
- други нужни методи за жизнен цикъл на обекта
- функция, която да извежда стринга в декомпресиран вид на текстов изходен поток
- length() – връща дължината на декомпресирания низ
- operator[] – достъп до посочен символ на низа, индексирането е според декомпресирания низ. Позволява промяна.
- operator+ за конкатенация два такива низа
- функция splice(int start, int length) – изтрива част от представения низ; премахват се length на брой елементи, като се започне от елемента на позиция start, индексирането е според декомпресирания низ
- функция insert(const RLEString& rles, int pos) – вмъква един RLE низ на позиция pos в друг RLE низ, индексирането е според декомпресирания низ.

Задача 2. Напишете програма, която прочита от клавиатурата символен низ, описващ дърво в следния формат:

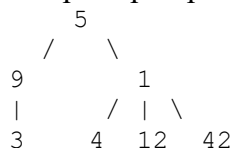
(<стойност в корена> {<наследници>})

Всеки възел в дървото може да има произволен брой наследници.

След като създаде дървото от прочетения низ, програмата трябва да изведе броя на онези елементи на дървото, които са по-малки от сумата на числата във всички възли намиращи се под тях в дървото. Листата нямат наследници, от които да са по-малки или по-големи.

След това трябва да се премахнат всички елементи, които са по-големи от сумата на числата във всички възли намиращи се под тях в дървото. При премахване на елемент неговите деца заемат мястото му. Единствено при премахване на корена, той трябва да се замени с първия от наследниците си, а останалите остават под него.

Например дървото



се представя като

(5 { (9 { (3 { }) }) } (1 { (4 { }) (12 { }) (42 { }) }) }))

За него само елементите 5 и 1 са с търсеното свойство, защото:

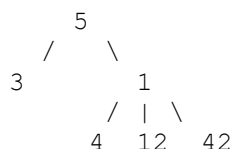
$$1 < 4 + 12 + 42$$

$$5 < 9 + 1 + 3 + 4 + 12 + 42$$

$9 > 3$ -- не изпълнява свойството

листата нямат наследници, от които да са по-малки

След премахването трябва да се получи



Позволява се използване на `std::string` и `std::vector`.